

the size of the figure to be generated, ensuring it's large enough to display the plot clearly. This helps us visualise the data effectively, making it easier to interpret the clusters.

```
# Visualize the clusters
plt.figure(figsize=(10, 6))
```

We plot the preferences of male and female customers on the same graph. For male preferences, we use blue markers, while for female preferences, we use red markers. By calling `plt.scatter()`, we create a scatter plot where the x-coordinate represents cappuccino preferences and the y-coordinate represents filter coffee preferences. This allows us to visualise the clustering of preferences for both genders simultaneously.

```
# Plot male preferences
plt.scatter(preferences[gender_labels ==
'Male', 0], preferences[gender_labels ==
'Male', 1], color='blue', label='Male')
```

```
# Plot female preferences
plt.scatter(preferences[gender_labels ==
'Female', 0], preferences[gender_labels ==
'Female', 1], color='red',
label='Female')
```

We plot the cluster centres on the scatter plot to visualise their positions in the feature space. Using the 'cluster_centers_' attribute obtained from the K-means model, we extract the coordinates of the cluster centres. Then, we plot these points on the existing scatter plot using black 'x' markers to distinguish them from the

customer preference data points. This helps us understand the centroids of each cluster and their relationship with the data points.

```
# Plot cluster centers
plt.scatter(cluster_centers[:, 0],
cluster_centers[:, 1], marker='x',
color='black', label='Cluster Centers')
```

We set the labels for the x-axis and y-axis of our plot using `plt.xlabel()` and `plt.ylabel()`, respectively. The x-axis is labelled as 'Cappuccino preference' and the y-axis as 'Filter coffee preference'. Additionally, we set the title of our plot to 'Clustering of male and female preferences for cappuccino and filter coffee' using `plt.title()`. To distinguish between male and female preferences, we add a legend using `plt.legend()`, which assigns the colours blue and red to male and female data points, respectively. Finally, we display the plot using `plt.show()`, ensuring it includes a grid for better visualisation clarity.

```
# Set labels and title
plt.xlabel('Cappuccino Preference')
plt.ylabel('Filter Coffee Preference')
plt.title('Clustering of Male and Female
Preferences for Cappuccino and Filter
Coffee')
```

```
# Add legend
```

```
plt.legend()

# Show plot
plt.grid(True)
plt.show()
```

With its simplicity, versatility, and rich ecosystem, Python continues to be a preferred choice for practitioners seeking to harness the power of cluster analysis for real-world applications. As organisations continue to grapple with the challenges posed by Big Data, Python's role in cluster analysis is poised to grow even further. Its scalability, coupled with the availability of distributed computing frameworks such as Dask and Spark, equips practitioners with the tools needed to tackle large-scale clustering tasks efficiently. This scalability is particularly crucial in domains such as genomics and bioinformatics, where datasets can be massive and computational resources are often limited.

Furthermore, Python's vibrant community and active development ensure that practitioners have access to the latest advancements and best practices in cluster analysis. Online forums, community-driven tutorials, and open source contributions foster collaboration and knowledge-sharing, enabling practitioners to stay abreast of emerging trends and techniques in the field. **END** 

 **By: Dr Chinmoy Kumar**

The author is engaged as an associate professor in business analytics at SCMS, Dayananda Sagar University, Bengaluru. His research and teaching interests revolve around emerging technologies, business models, e-business competitiveness and e-governance for development.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU